Formal Verification of the ODC Handshake: A Tamarin Model of an Authenticated Controller

ABDELKADER OUDA

Department of Electrical and Computer Engineering, Western University, London, Ontario, Canada.

Artifact: doi:10.5281/zenodo.23289653
Source: github.com/UWO-ECE-Software-Engineering/odc_tamarin

October 10, 2026

Abstract

We present a symbolic model, in the Tamarin prover, of the handshake of the Open Driver Communication (ODC) protocol. ODC is the authenticated channel between a cryptographic-resource controller and the adapter of a hardware security module (HSM) in a heterogeneous, multi-vendor HSM control plane. The handshake combines signed Diffie-Hellman key agreement with negotiation between a hybrid (classical plus post-quantum) mode and a classical mode, and is followed by an authenticated operation request. For an unbounded number of sessions, under a Dolev-Yao network adversary who may also reveal any long-term key, Tamarin automatically proves seven lemmas: executability, session-key secrecy with forward secrecy from both sides, mutual injective agreement on the full transcript, resistance to classical-mode downgrade, and authentic, replay-free request execution. The complete analysis takes about 7-12 s on a laptop. As a mutation check, removing the offered-mode list from the signed transcript yields a downgrade attack that Tamarin finds automatically; the counterexample is shown. The model is a first step toward verifying the controller-device protocols of a vendor-independent, crypto-agile HSM management architecture.

Contents

Introduction **3**

1 The ODC handshake **3**

2 Model and threat model **4**

3 Security properties **4**

4 Results **5**
4.1 Mutation check: removing offer binding 5

5 Scope and limitations **7**

6 Next steps **7**

7 Reproducibility **7**

Introduction

Organizations increasingly operate HSMs from several vendors, on premises and in the cloud, while preparing to migrate from classical public-key algorithms to the post-quantum standards ML-KEM and ML-DSA [6]. A control plane that manages such a fleet as one resource needs an authenticated protocol between its controllers and each device adapter. That protocol must also negotiate which algorithm mode a session uses, because during migration only some devices support post-quantum mechanisms. Negotiation creates a classic risk: an active attacker may strip the stronger option and push two capable parties onto the weaker one, a downgrade attack [5]. ODC (Open Driver Communication) is the controller-device protocol of the proposed architecture. This report models its handshake and first request in Tamarin [1] and states, proves, and stress-tests the security properties the rest of the architecture relies on. The contribution is deliberately narrow: a precise, machine-checked specification of the session layer, released openly so that later layers (key lifecycle, migration policy) can build on it. Related formal work on HSMs verifies the configuration or key-management policy of a single device [7, 8]; it does not address the authenticated, mode-negotiating channel between a controller and many devices.

1 The ODC handshake

A controller C and a device adapter D each hold a long-term signing key whose public half was enrolled in advance (a trust anchor). The controller always offers both modes, preferring hybrid. Writing g^x for a Diffie-Hellman share, n_C and n_D for fresh nonces, and $sig_A()$ for A's signature:

1. $C \to D : C, D, n_C, g^x, offer$ offer = $\langle hybrid, classical \rangle\rangle$
2. $D \to C : n_D, g^y, v, sig_D(\langle ODC_D, C, D, n_C, n_D, g^x, g^y, offer, v\rangle)$
3. $C \to D : sig_C(\langle ODC_C, C, D, n_C, n_D, g^x, g^y, offer, v\rangle), k = h(g^x y, n_C, n_D)$
4. $C \to D : senc(\langle REQ, rid, op, kh\rangle, k)$ D executes each rid at most once

Here v is the mode the device chooses, rid a fresh request identifier, op an operation and kh a key handle. Three design decisions matter for the properties below:

- Full-transcript signatures. Each signature covers both identities, both nonces, both shares, the offer and the chosen mode, with a role tag (ODC_D or ODC_C). This binds the negotiation into the authentication, in the style of signed Diffie-Hellman protocols such as SIGMA [3].
- Device-side mode policy. A hybrid-capable device chooses hybrid whenever it is offered and falls back to classical only when hybrid is absent. A legacy device supports classical only.
- Request identifiers. The device keeps the identifiers it has executed and rejects repeats. This is the anti-replay mechanism for requests within a session.

2 Model and threat model

The Tamarin theory uses the built-in equational theories for Diffie-Hellman, signatures, symmetric encryption and hashing, so cryptography is perfect. The network is a Dolev-Yao adversary [4] who reads, drops, reorders, replays and forges messages. A reveal rule lets the adversary obtain the long-term key of any controller or device at any time; lemmas state exactly which reveals they tolerate. The number of sessions, controllers and devices is unbounded.

- Enrollment. Rules register a controller, a hybrid-capable device or a legacy device; each identity registers once (a restriction).
- Roles. C_1 starts a session; D_1_Hybrid_Pick_Hybrid, D_1_Hybrid_Fallback and D_1_Legacy are the three device responses; C_2 and D_2 verify the signatures and accept; C_Request and D_Execute send and execute one operation.
- Restrictions. Signature checks are equality restrictions on $verify(.) = true$. Fallback applies only when the first offered element is not hybrid. The anti-replay check is a uniqueness restriction on (device, rid).
- Abstractions. Each mode is a constant; the model does not represent the internals of ML-KEM or ML-DSA, and treats a hybrid session as one agreed mode label. Long-term keys are ordinary signature keys.

3 Security properties

Table 1 lists the seven lemmas in plain language. Two are shown below in Tamarin syntax; the others follow the same pattern and are in the released theory.

lemma no_downgrade:

$$
\begin{aligned}
&\text{"}All\ C\ D\ nc\ nd\ offer\ v\ k\ \#\ i\ \#\ j.\\
&\quad C_Accept(C,\ D,\ nc,\ nd,\ offer,\ v,\ k)\ @\ \#i\ \&\ HybridDevice(D)\ @\ \#j\\
&\quad \&\ not\ (Ex\ \#r.\ RevLtk(D)\ @\ \#r)\ \Rightarrow\ v =' hybrid'\text{"}
\end{aligned}
$$

lemma key_secrecy_pfs:

$$
\begin{aligned}
&\text{"}All\ C\ D\ nc\ nd\ offer\ v\ k\ \#i.\\
&\quad C_Accept(C,\ D,\ nc,\ nd,\ offer,\ v,\ k)\ @\ \#i.\\
&\quad \&\ not\ (Ex\ \#r.\ RevLtk(D)\ @\ \#r\ \&\ \#r\ \prec\ \#i)\\
&\quad \&\ not\ (Ex\ \#r.\ RevLtk(C)\ @\ \#r\ \&\ \#r\ \prec\ \#i)\\
&\quad \Rightarrow\ not\ (Ex\ \#j.\ K(k)\ @\ \#j)\text{"}
\end{aligned}
$$

Forward secrecy follows from the ordering constraint $\#r \prec \#i$: compromise of either long-term key after the controller accepts does not reveal the session key. The agreement lemmas are injective in the sense of Lowe [2]: each acceptance corresponds to a unique matching run of the peer on the same transcript, which rules out replayed and reflected sessions.

4 Results

All seven lemmas are verified automatically, with no manual guidance, by Tamarin 1.10.0 with Maude 3.5. Proof sizes are the number of proof steps Tamarin reports. The full analysis took 7.1 s on the author's Windows 11 laptop (WSL, Ubuntu) and 11.8 s on a Linux container.

Table 1: Verification results for the security lemmas.

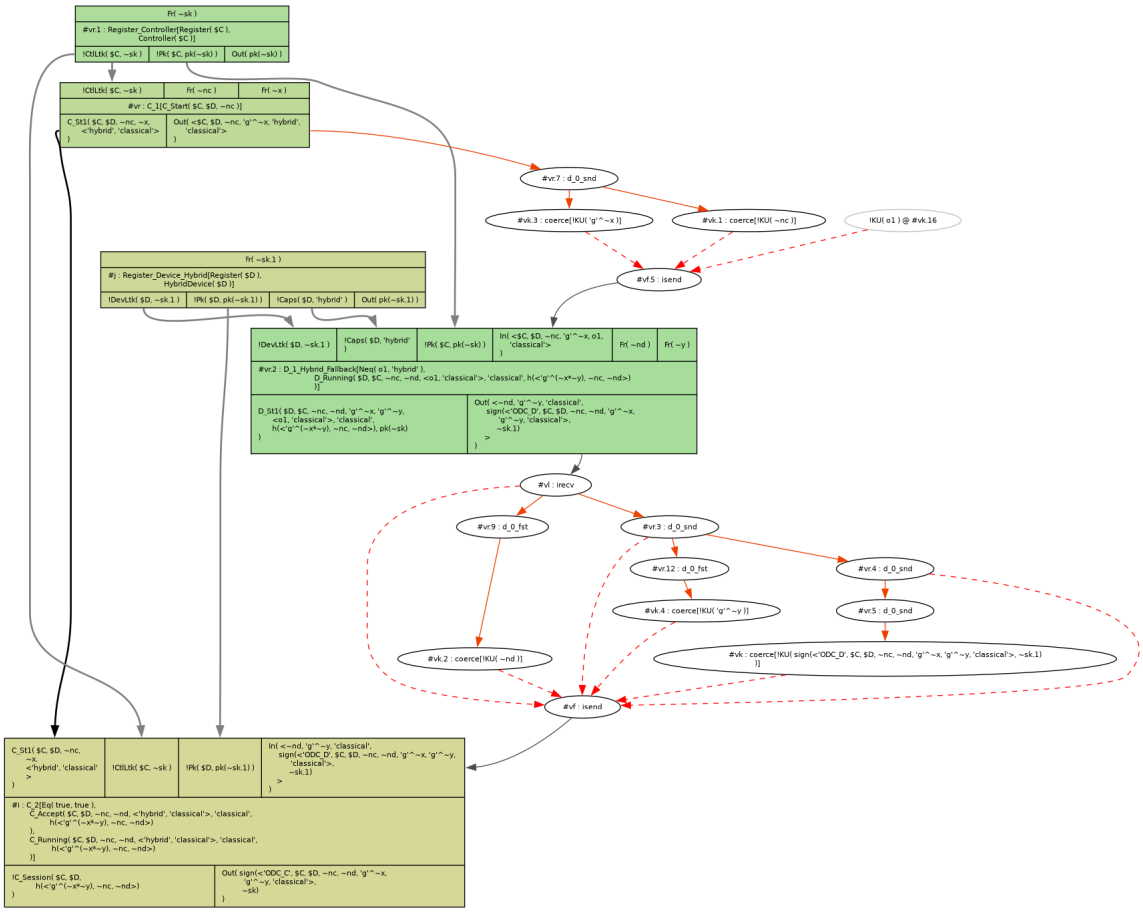
Lemma	Property	Result	Steps
`executable`	An honest run reaches execution of an operation on a hybrid device (sanity check)	verified	10
`key_secrecy_pfs`	Session key secret; only compromise before acceptance breaks it	verified	25
`device_key_secrecy`	Session key secret from the device's view (reused as a helper)	verified	47
`controller_injective_agreement`	Controller authenticates the device on the full transcript, one-to-one	verified	36
`device_injective_agreement`	Device authenticates the controller on the full transcript, one-to-one	verified	156
`no_downgrade`	No classical session with a hybrid-capable, uncompromised device	verified	9
`request_injective_agreement`	Each executed request was sent in that session by the controller, and runs once	verified	73

4.1 Mutation check: removing offer binding

A verified property is only informative if it can fail. The variant ODC_Handshake_NoOfferBinding.spthy is identical except that the offer list is omitted from both signed transcripts. Tamarin falsifies no_downgrade on this variant in a 10-step trace (Figure 1):

- The controller offers $\langle hybrid, classical \rangle$.
- The adversary replaces the first element with a value $o1$ of its own, so the device receives $\langle o1, classical \rangle$.
- The hybrid-capable device, seeing no hybrid option, takes the fallback rule and chooses classical. It signs a transcript that no longer contains the offer.
- The controller verifies a valid signature and accepts the classical session. No key was revealed.

In the full model the device's signature covers the offer it received, so the controller detects the substitution and rejects. The mutation therefore confirms that offer binding is the mechanism that provides downgrade resistance, mirroring the transcript-binding lessons of earlier downgrade attacks on deployed protocols [5].



Counterexample found by Tamarin when the offered-mode list is omitted from the device's signed transcript: an attacker strips 'hybrid' from the offer, the hybrid-capable device falls back to classical, and the controller accepts. Binding the offer in the signature (ODC_Handshake.spthy) removes the attack; no_downgrade is then verified.

Figure 1: *Tamarin counterexample to no_downgrade for the variant without offer binding. The adversary-supplied value $KU(o1)$ replaces 'hybrid' in the offer; D_1_Hybrid_Fallback selects 'classical'; C_2 accepts.*

5 Scope and limitations

- Symbolic, design-level. Results hold for the protocol design under perfect cryptography. They say nothing about vendor firmware, side channels or a particular implementation.
- Two modes. The model covers hybrid and classical. The four migration states of the target architecture (classical-only, hybrid-required, PQC-preferred with approved fallback, PQC-only) are not yet modelled.
- Anti-replay as an assumption. Request uniqueness is stated as a restriction, so the replay result shows that the protocol is correct given a device that enforces unique identifiers, not how the device stores them.
- No key lifecycle. Operations on keys (generation, rotation, revocation, recovery) and multiple controllers sharing key state are outside this model.

6 Next steps

Planned extensions are: the four migration states with policy-dependent fallback; key-lifecycle actions with a lemma that no operation executes on a revoked key handle after revocation reaches the device; a cross-check of the handshake in ProVerif; and TLA+ models of multi-controller key-state consistency under failure and partition, for which this session-layer model supplies the authenticated-channel assumption.

7 Reproducibility

The theory files, the mutation variant, installation and run scripts (Linux and Windows 11 via WSL) and the outputs are archived at doi:10.5281/zenodo.23289653 under the MIT licence. To reproduce:

```
bash install_tamarin_wsl.sh     # Tamarin 1.10.0 + Maude 3.5
bash run_proofs.sh              # proves 7 lemmas; falsifies no_downgrade
                                  on the variant
tamarin-prover interactive.     # browse proofs and the attack graph
```

References

1. S. Meier, B. Schmidt, C. Cremers, and D. Basin. The TAMARIN prover for the symbolic analysis of security protocols. CAV 2013, LNCS 8044, pp. 696–701, 2013.

2. G. Lowe. A hierarchy of authentication specifications. Proc. 10th IEEE Computer Security Foundations Workshop, pp. 31–43, 1997.

3. H. Krawczyk. SIGMA: The 'SIGn-and-MAc' approach to authenticated Diffie–Hellman and its use in the IKE protocols. CRYPTO 2003, LNCS 2729, pp. 400–425, 2003.

4. D. Dolev and A. Yao. On the security of public key protocols. *IEEE Transactions on Information Theory*, 29(2):198–208, 1983.

5. K. Bhargavan, C. Brzuska, C. Fournet, M. Green, M. Kohlweiss, and S. Zanella-Béguelin. Downgrade resilience in key-exchange protocols. IEEE Symposium on Security and Privacy, pp. 506–525, 2016.

6. National Institute of Standards and Technology. FIPS 203 (ML-KEM) and FIPS 204 (ML-DSA), August 2024.

7. R. Focardi and F. L. Luccio. A formally verified configuration for hardware security modules in the cloud. Proc. ACM CCS, pp. 412–428, 2021.

8. M. Busi, R. Focardi, S. Gul, and F. L. Luccio. Automated analysis of key management policies. Joint National Conference on Cybersecurity (ITASEC & SERICS 2025), CEUR Workshop Proceedings, vol. 3962, 2025.